

Algoritmi e strutture dati - esame del 1/7/2019

Svolgimenti

Cognome e nome: _____ Matr. _____

Esercizio 1. (Punti 3) Si considerino le due funzioni $f(n) = n$ e $g(n) = n^{1+\sin n}$. Si dimostri che le due funzioni non sono confrontabili asintoticamente, ossia che $f(n) \notin O(g(n))$ e che $f(n) \notin \Omega(g(n))$.

Soluzione 1. Dato che $\sin n$ oscilla infinite volte tra -1 e 1 , $1 + \sin n$ oscilla tra 0 e 2 , e quindi $g(n) \approx n^0 = 1$ e $g(n) \approx n^2$ per infiniti valori di n . Se fosse $f(n) \in O(g(n))$ allora dovremmo avere anche $f(n) \in O(\min(1, n^2)) = O(1)$; analogamente $f(n) \in \Omega(g(n))$ implica $f(n) \in \Omega(\max(1, n^2)) = \Omega(n^2)$. Ma $f(n) = n \notin O(1)$ e $n \notin \Omega(n^2)$, onde la tesi.

Esercizio 2. (Punti 6) Si consideri l'algoritmo seguente:

GOO(n)

Pre: $n > 0$ numero intero

if $n = 1$ **then**

return 100

else

return GOO($n - 2$) + GOO($n - 2$)

Senza far uso del master theorem, si trovi il Θ della sua complessità in funzione di n ; quindi si scriva lo pseudo-codice di un algoritmo equivalente, ma di complessità asintoticamente migliore.

Soluzione 2. La relazione di ricorrenza dell'algoritmo GOO è $T(n) = 2T(n-2) + 1$, il cui sviluppo produce l'equazione:

$$T(n) = 2^k T(n - 2k) + \sum_{i=0}^{k-1} 2^i, \quad k \leq n/2.$$

Ponendosi nell'estremo otteniamo:

$$T(n) = 2^{n/2} \cdot \Theta(1) + \sum_{i=0}^{n/2-1} 2^i = \Theta(2^{n/2}) + 2^{(n/2)+1} - 1 = \Theta(2^{n/2})$$

Per ottenere un algoritmo migliore basta non calcolare due volte GOO($n - 2$) in ogni chiamata ricorsiva, ad esempio ponendo:

FASTGOO(n)

Pre: $n > 0$ numero intero

if $n = 1$ **then**

return 100

else

return 2 · FASTGOO($n - 2$)

La relazione di ricorrenza di FASTGOO è infatti $T(n) = T(n-2) + 1$, che facilmente si vede essere $\Theta(n)$.

Esercizio 3. (Punti 7) Si consideri il seguente algoritmo:

```

BAMBY( $A[0..n-1]$ )
  for  $i \leftarrow 0$  to  $n-1$  do
    if  $A[i] \bmod 2 = 1$  then
       $b \leftarrow A[i]$ 
      for  $j \leftarrow i$  downto 1 do
         $A[j] \leftarrow A[j-1]$ 
       $A[0] \leftarrow b$ 
  return  $A$ 

```

- Cosa fa BAMBY? Si descriva *brevemente* cosa fa l'algoritmo, non come lo fa.
- Qual è l'invariante del ciclo più esterno?
- Qual è la complessità di BAMBY in termini di Θ ?
- Si scriva lo pseudo-codice di un algoritmo equivalente a BAMBY, ma di complessità asintoticamente migliore.

Soluzione 3. BAMBY permuta gli elementi di $A[0..n-1]$ in modo che tutti i dispari precedano i pari. L'invariante del ciclo più esterno stabilisce che questo è vero nel sottoarray $A[0..i-1]$, che espresso formalmente (ma anche la versione informale è accettabile) risulta:

$$\exists k \in 0..i \ (\forall x \in A[0..k-1] \ x \bmod 2 = 1 \wedge \forall y \in A[k..i-1] \ y \bmod 2 = 0).$$

La complessità è data dal numero di assegnazioni $A[j] \leftarrow A[j-1]$ nel caso peggiore, che si verifica quando $A[i] \bmod 2 = 1$ è vera per ogni i , ossia quando l'array ha tutti gli elementi dispari. Queste risultano pari a

$$\sum_{i=0}^{n-1} (n-i) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Un algoritmo migliore di BAMBY si ottiene come una variante dell'algoritmo della bandiera che opera su due partizioni invece di tre:

```

FASTBAMBY( $A[0..n-1]$ )
   $i \leftarrow 0, j \leftarrow 0$ 
  while  $i < n$  do
    if  $A[i] \bmod 2 = 1$  then
      scambia  $A[i]$  con  $A[j]$ 
       $j \leftarrow j+1$ 
     $i \leftarrow i+1$ 
  return  $A$ 

```

L'invariante del **while** è che $A[0..j-1]$ è costituito da dispari e $A[j..i-1]$ da pari. FASTBAMBY ha complessità $\Theta(n)$. Un'altra possibilità è modificare l'algoritmo PIVOT del QUICKSORT, anch'esso di complessità lineare.

Esercizio 4. (Punti 4) Si consideri una tabella hash di dimensione $m = 11$ e la sequenza di chiavi:

35, 83, 57, 26, 15, 63, 97, 46.

Si inseriscano tutte le chiavi nell'ordine dato utilizzando la tecnica dell'indirizzamento aperto con doppio hash:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

dove $h_1(k) = k \bmod m = k \bmod 11$ e $h_2(k) = 1 + k \bmod (m-1) = 1 + k \bmod 10$.

Soluzione 4. Costruiamo una tabella per calcolare i valori successivi di $h(k, i)$ in funzione delle locazioni precedentemente occupate:

k	$h_1(k)$	$h_2(k)$	i	$h(k, i)$	
35	2	6	0	2	ok
83	6	4	0	6	ok
57	2	8	0	2	occupato
57	2	8	1	10	ok
26	4	7	0	4	ok
15	4	6	0	4	occupato
15	4	6	1	10	occupato
15	4	6	2	5	ok
63	8	4	0	8	ok
97	9	8	0	9	ok
46	2	7	0	2	occupato
46	2	7	1	9	occupato
46	2	7	2	5	occupato
46	2	7	3	1	ok

La tabella così ottenuta risulta:

0	1	2	3	4	5	6	7	8	9	10
	46	35		26	15	83		63	97	57

Esercizio 5. (Punti 6) Si dia lo pseudo-codice dell'algoritmo SUMBRANCH(T) che, dato un albero k -ario $T \neq \emptyset$ con etichette intere realizzato con puntatori *child* e *sibling*, aggiunge al fondo di ogni ramo di T un nuovo nodo avente per etichetta la somma delle etichette sul ramo cui è aggiunto. L'algoritmo deve avere complessità $O(n)$ nella cardinalità di T .

Suggerimento: si utilizzi una funzione ausiliaria SUMBRANCH-AUX(T, s) che accumuli in s la somma delle etichette sul ramo esplorato.

Soluzione 5. La soluzione qui proposta utilizza una funzione ausiliaria SUMBRANCH-AUX(T, s) che consiste in una DFS modificata in modo da accumulare in s la somma delle etichette sul ramo esplorato:

```

SUMBRANCH-AUX( $T, s$ )
  if  $T.child = nil$  then
     $T.child \leftarrow$  un nuovo nodo con etichetta  $s + T.key$ 
  else
     $s \leftarrow s + T.key$ 
     $C \leftarrow T.child$ 
    while  $C \neq nil$  do
      SUMBRANCH-AUX( $C, s$ )
       $C \leftarrow C.sibling$ 

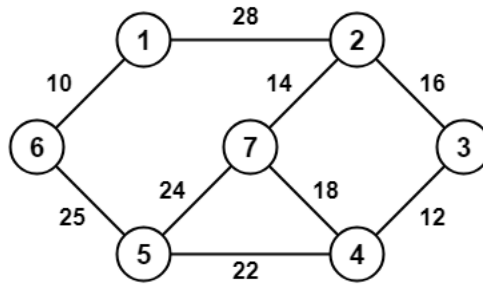
```

Ne segue che SUMBRANCH(T) sarà costituito dalla sola istruzione SUMBRANCH-AUX($T, 0$). La complessità è $O(n)$ essendo SUMBRANCH-AUX una DFS.

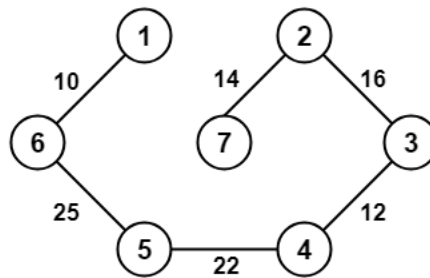
Esercizio 6. (Punti 5) Si riporti lo pseudo-codice dell'algoritmo TREESUCCESSOR(p) che, dato un puntatore $p \neq nil$ ad un vertice di un albero binario di ricerca T realizzato con puntatori *left*, *right* e *parent*, ritorna un puntatore al vertice contenente il successore di $p.key$ in T se esiste, *nil* altrimenti.

Soluzione 6. Vedi Cormen, par. 12.2.

Esercizio 7. (Punti 5) Si calcoli l'MST (l'albero minimo ricoprente) del grafo pesato (G, w) sotto riportato utilizzando l'algoritmo di Prim a partire dal vertice 1:



Soluzione 7. Visto che non ci sono due archi in G di peso uguale, una volta fissato il vertice di partenza l'MST risultante dall'esecuzione di Prim è univocamente determinato; nel caso in cui si parta da 1 l'MST risulta:



dove gli archi sono stati scelti nell'ordine:

$(1, 6), (6, 5), (5, 4), (4, 3), (3, 2), (2, 7).$